TECHNICAL DOCUMENTATION

RAG-Based Customer Support Assistant

with LangGraph and Human-in-the-Loop

Prepared by: Kartik Patade

1. Introduction

1.1 What is RAG

RAG stands for Retrieval-Augmented Generation. It is a technique where a language model is given relevant information retrieved from an external source before generating a response. Instead of relying purely on what the model learned during training, it looks up facts from a document (or database) at the time of the query.

The basic idea is straightforward: when someone asks a question, you first search through a knowledge base for relevant content, then give that content to the language model along with the question, and let it produce an answer grounded in that content.

1.2 Why RAG is Needed

Language models have a knowledge cutoff. They cannot access company-specific documents, internal policies, product manuals, or real-time information. For a customer support use case, this is a significant limitation.

RAG solves this by combining the reasoning and language capabilities of an LLM with fresh, domain-specific content from a document. The result is a system that can give accurate, contextually relevant answers based on actual support documentation rather than generic training data.

1.3 Use Case Overview

This project implements a customer support assistant that reads from a PDF knowledge base. A customer types a question. The system finds the most relevant sections of the document, passes them to a language model, and returns a clear answer. If the system is not confident, it flags the query for a human support agent to handle.

2. System Architecture Explanation

2.1 Overview

The system has two phases. The ingestion phase processes the PDF and builds the vector store. The query phase handles incoming user questions using the stored embeddings.

2.2 Ingestion Phase

The PDF is loaded using PyPDFLoader. The extracted text is split into chunks of 500 characters with 50-character overlaps. Each chunk is then converted into a vector using the all-MiniLM-L6-v2 embedding model. These vectors are stored in ChromaDB on disk.

This phase runs once when the app is first set up, or when the knowledge base is updated. It does not need to run for every user query.

2.3 Query Phase

When a user sends a query, it is passed to the LangGraph workflow. Inside the graph, the query is first embedded using the same embedding model. The embedding is then compared against all stored vectors in ChromaDB using cosine similarity. The top 3 matching chunks are returned.

These chunks are combined with the original query into a prompt. The prompt is sent to the Gemini 1.5 Flash model via API. The response is then evaluated by the routing logic.

2.4 Component Interactions

LangChain acts as the glue layer. It provides the document loaders, text splitters, and retrieval chains. LangGraph sits on top of LangChain and defines the workflow as a stateful graph. ChromaDB is a standalone component that handles storage and search. The LLM (Gemini) is called via the LangChain Google GenAI integration.

3. Design Decisions

3.1 Chunk Size Choice

A chunk size of 500 characters was chosen after some experimentation. Smaller chunks (200-300 characters) were too short and often lacked enough context for the LLM to generate a useful answer. Larger chunks (1000+ characters) sometimes included too much unrelated content, which confused the model.

500 characters tends to be around 3-5 sentences, which is a natural unit of information for a support document.

3.2 Embedding Strategy

The all-MiniLM-L6-v2 model was selected because it runs locally, requires no API key, and produces 384-dimensional embeddings that are fast to compute and store. For a project of this scale, a heavier model like text-embedding-3-large would be overkill and would add cost. MiniLM works well for semantic similarity tasks.

3.3 Retrieval Approach

Top-k retrieval with k=3 was used. Returning more chunks increases context but also adds noise. Three chunks gives the LLM enough material to work with without overwhelming the prompt or hitting token limits.

3.4 Prompt Design Logic

The prompt instructs the LLM to answer only from the provided context. It also tells the model to say 'I don't know' or 'I cannot find' if the context does not contain enough information. This is important because it makes the uncertainty detection in the routing layer reliable. If the model generates a confident-sounding but wrong answer, the routing logic cannot detect that.

```
Prompt template:
'You are a helpful customer support assistant.'
'Use ONLY the context below to answer the question.'
'If the context does not have enough information, say so.'
'Context: {context}'
'Question: {query}'
'Answer:'
```

4. Workflow Explanation

4.1 LangGraph Usage

LangGraph is used to define the query workflow as a directed graph. Each step in the process is a node. Data flows between nodes via a shared state dictionary. Edges define what happens after each node, and conditional edges allow branching.

The main reason for using LangGraph rather than a plain function chain is that it makes the control logic explicit. You can see exactly when and why the system routes to escalation. It also makes the workflow easier to extend later.

4.2 Node Responsibilities

retrieve_and_generate

This node does two things: retrieves relevant chunks and generates the LLM response. It takes the query from the state, calls the retriever, formats the prompt, and calls the LLM. It writes context and response back into the state.

route_response

This node reads the response and decides whether to escalate. It checks for the uncertainty phrases and for an empty retrieved docs list. It sets the escalate flag in the state.

output_node

This node reads the final state and either prints the answer or triggers the HITL handler based on the escalate flag.

4.3 State Transitions

State starts as: { query: 'user question', context: '', response: '', escalate: False }

After retrieve_and_generate: context and response are filled in.

After route_response: escalate is set to True or False.

After output_node: the result is displayed and the graph ends.

5. Conditional Logic

5.1 Intent Detection

The system does not use a dedicated intent classifier. Instead, it uses two simple signals: the content of the LLM response (does it contain uncertainty phrases?) and whether any relevant chunks were found at all.

A few keywords in the query (like 'legal' or 'billing dispute') are also flagged as complex queries that should always escalate, regardless of whether the model produced an answer.

5.2 Routing Decisions

Confident answer: retrieved docs are not empty AND response has no uncertainty phrases AND query has no complex keywords. Route to output.

Escalate: any of the above conditions fails. Route to HITL handler.

6. HITL Implementation

6.1 Role of Human Intervention

HITL (Human-in-the-Loop) exists to handle queries that the automated system cannot reliably answer. In a real deployment, this could be a Tier 2 support agent who reviews flagged tickets and responds manually.

In this implementation, HITL is simulated. The system logs the escalated query and prints a message. The idea is that a human would check the escalation log and follow up directly with the customer.

6.2 Benefits

- Prevents incorrect or confusing answers from reaching the customer
- Gives visibility into what the system cannot handle, which can guide future improvements to the knowledge base
- Builds user trust by routing genuinely complex issues to humans

6.3 Limitations

- In this implementation, there is no actual communication channel to a human agent
- The uncertainty detection is based on simple phrase matching, so it can miss cases where the model gives a wrong but confident-sounding answer
- There is no feedback loop from the human response back into the system for learning

7. Challenges and Trade-offs

7.1 Retrieval Accuracy vs Speed

Cosine similarity search is fast and works well, but it can miss relevant chunks if the query phrasing is very different from the document language. A hybrid approach using both dense

(embedding) and sparse (BM25 keyword) retrieval would improve accuracy but adds complexity. For this project, dense-only retrieval was kept.

7.2 Chunk Size vs Context Quality

Smaller chunks make retrieval more precise but give the LLM less context. Larger chunks give more context but retrieve less specifically. There is no perfect answer here. The 500/50 setting worked reasonably well for the test document.

7.3 Cost vs Performance

Using a local embedding model keeps costs at zero. The only cost is the Gemini API calls, which have a free tier. For higher volumes, this would need to be monitored. Alternatively, a locally hosted LLM like Ollama with Mistral could completely eliminate API costs, at the cost of needing a more powerful machine.

7.4 Latency and Performance Bottlenecks

During testing, the system exhibited noticeable latency between query input and response generation. This is attributed to three primary factors:

- **Local Embedding Overhead**: Utilizing the all-MiniLM-L6-v2 model locally on a CPU introduces a computation delay for every query. While cost-effective, it lacks the speed of dedicated embedding APIs or GPU-accelerated processing.
- **API Round-Trip Latency**: As the system uses the Google Gemini 1.5 Flash API, each request is subject to network travel time and remote server processing.
- **Free-Tier Rate Limiting**: The use of the Gemini Free Tier resulted in occasional 429 TooManyRequests errors during rapid testing, requiring the implementation of manual pacing between queries.

7.5 Retrieval Accuracy vs. Extraction Quality

- **PDF Formatting Issues**: Text extraction via PyPDFLoader occasionally resulted in irregular character spacing (e.g., "F or W indows 7"). While this did not break the logic, it slightly degraded the readability of the LLM-generated output.
- **Context Window Constraints**: We utilized top-k=3 retrieval to balance context depth with processing speed. Increasing k would provide more information but significantly increase the tokens sent to Gemini, further slowing down the response time.

7.6 Architectural Trade-offs

- **Single-Document Focus**: To ensure high retrieval precision for the internship project, the system was limited to a single PDF knowledge base. This avoids "noise" from unrelated documents but limits the assistant's overall utility.
- **Stateless Conversations**: The decision to exclude conversation memory makes the system faster per query but prevents users from asking follow-up questions like "Can you explain that further?".

8. Testing Strategy

8.1 Testing Approach

Testing was done manually with a set of sample queries. Three categories were tested: direct questions that should be answered from the document, ambiguous questions where the document has partial information, and off-topic questions that should trigger escalation.

8.2 Sample Queries

Queries expected to be answered:

- What is your refund policy?
- How do I contact support?
- What documents are required to open an account?

Queries expected to escalate:

- I want to file a legal complaint about a billing error.
- What is the meaning of life?
- Can you tell me what my account balance is right now?

The routing logic correctly escalated all three off-topic or complex queries and produced answers for all three document-based queries during testing.

9. Future Enhancements

9.1 Multi-Document Support

Currently the system loads a single PDF. Extending it to support multiple documents (or a whole folder of PDFs) would make it more useful. ChromaDB supports multiple collections, so each document type could have its own collection, or all could be merged into one.

9.2 Feedback Loop

After a human handles an escalated query, their response could be added back to the knowledge base as a new chunk. Over time, this would reduce escalations for similar questions. This would require a simple admin interface and a re-indexing step.

9.3 Memory Integration

Currently each query is treated independently. Adding conversation memory (using LangChain's ConversationBufferMemory or a similar approach) would allow follow-up questions like 'Can you explain that further?' to work correctly.

9.4 Deployment

The current system is CLI-based. Wrapping it with a FastAPI backend and a simple frontend (or integrating it into a chat widget) would make it usable for actual customers. Docker would help package all dependencies cleanly.